

Guida Utente alla

Procedura di anonimizzazione dei dati con il *privacy filter* di OpenAI

in Windows

Antonio Piemontese

2 giugno 2026

Il **Privacy Filter di OpenAI** è uno strumento *open-source* (sotto licenza Apache 2.0) pensato per **intercettare, riconoscere e oscurare informazioni personali o potenzialmente sensibili** presenti in un testo prima che questo venga utilizzato in un processo di elaborazione automatica, ad esempio in un prompt destinato a un modello linguistico, in una pipeline di intelligenza artificiale, in un sistema RAG, in un dataset di addestramento, in un log applicativo. Il suo scopo principale è **ridurre il rischio** che dati identificativi, come nomi, indirizzi, email, date di nascita, numeri di telefono o altri riferimenti personali, vengano trasmessi o conservati inutilmente. Dal punto di vista interno, il filtro non si limita a una semplice ricerca meccanica di parole o pattern, ma analizza il testo nel suo contesto, individua le **porzioni che possono rappresentare dati personali e le sostituisce con etichette, mascheramenti o forme redatte**. In questo senso, il Privacy Filter si colloca tra gli strumenti di protezione preventiva del dato: non modifica la logica del documento né ne nasconde necessariamente il significato generale, ma interviene sugli elementi che potrebbero consentire l'identificazione di una persona, eventualmente di un'azienda, o la diffusione di informazioni riservate. Occorre tuttavia considerarlo come uno strumento di **mitigazione del rischio**, non come una garanzia assoluta di anonimizzazione, perché il riconoscimento automatico può sempre produrre omissioni o oscuramenti eccessivi, soprattutto quando il dato personale è implicito, ambiguo o dipendente dal contesto.

👉 Lo strumento OpenAI privacy filter è tecnologicamente **agnostico** perché produce file oscurati che possono essere dati in input a qualsiasi strumento o chatbot (non solo chatGPT).

Il modello è caratterizzato da 1.5B parametri totali ma solo 50M attivi, e finestra di contesto da 128.000 token che evita il chunking.

Legenda

- 👉 punto di attenzione, il "succo"
- 📌 nota / inizio di una nota
- ✅ punto risolto, positivo
- ❌ punto negativo, da evitare
- 🔴 problema
- ⚠️ attenzione, warning, allarme
- conseguenza, effetto, passo successivo

Strumenti necessari

- un terminale di Windows (il Prompt dei Comandi)
- Python
- PyTorch (un package Python per gestire reti neurali)

Eeguire i seguenti passi:

0. SE la cartella per la prova del privacy filter di OpenAI è **già stata creata**, occorre **cancellarla** preventivamente per avere **una situazione di partenza pulita**, in questo modo:

- supponiamo che la cartella da cancellare sia `C:\Utenti\<Utente>\privacy-filter` (oppure, al posto del generico `<Utente>`, il proprio specifico utente)
- posizionarsi sulla cartella **padre**, ad esempio con: `cd C:\Utenti\<Utente>` ¹
- eseguire il comando:

```
rmdir /s /q privacy-filter
```

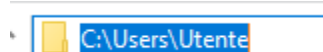
dove:

- `/s`: indica a Windows di rimuovere la cartella principale e *tutte le sottocartelle* al suo interno.
- `/q`: esegue l'operazione in modalità "silenziosa" (quiet). Non chiede conferma per ogni singolo file o cartella.

Attenzione:

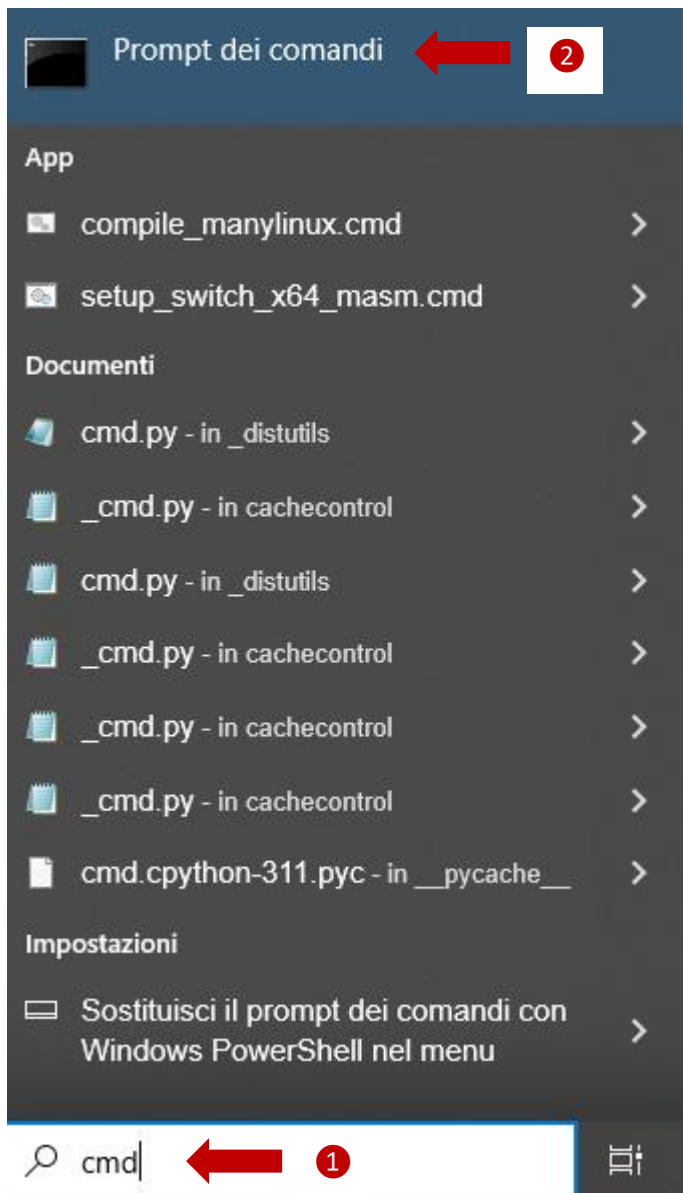
L'eliminazione di cartella tramite riga di comando è **definitiva**. I file rimossi in questo modo **non vanno nel cestino di Windows**, quindi fare molta attenzione a trovarsi nella cartella corretta prima di premere Invio.

¹ Oppure navigando con Explorer sino a `C:\Utenti\<Utente>`, poi selezionando la barra in alto con il cammino completo della cartella

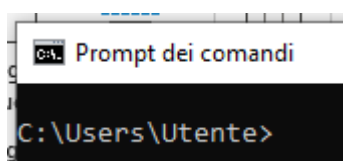


e poi digitando `cmd` + invio. Si aprirà una finestra `cmd` posizionata proprio su `C:\Utenti\<Utente>`.

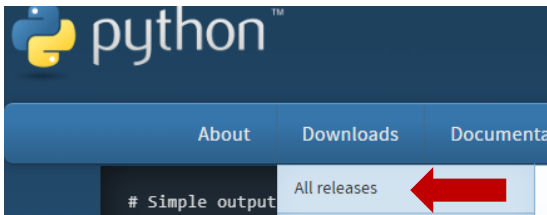
1. attivare un Prompt dei comandi di Windows, scrivendo "cmd" nella barra di ricerca di Windows in basso a sinistra dello schermo e poi selezionando la voce "Prompt dei comandi"



2. si apre una finestra nera con titolo "Prompt dei comandi", in genere impostata sulla cartella C:\Utenti\<Utente> (dove il generico <Utente> può essere stato sostituito dal vostro utente specifico, se precedentemente creato in Windows).



3. andare sul sito python.org e poi premere "Downloads" → "All releases"



4. guardare al centro dello schermo, sotto il grande pulsante giallo.



Trovare questa riga di testo:

*Looking for Python with a different OS? Python for **Windows**, Linux/Unix, macOS, Android, other.*

Cliccare sulla parola gialla **Windows**.

5. si aprirà una pagina intitolata "Python Releases for Windows".

Python Releases for Windows

- scorrere verso il basso fino a trovare la versione che desideriamo (ad oggi, **Python 3.12.9**).

[Python 3.12.9 - Feb. 4, 2025](#)

Note that Python 3.12.9 cannot be used on Windows 7 or earlier.

- [Download Windows installer \(64-bit\)](#)

- sotto il titolo della versione scelta, cliccare sul link **Download Windows installer (64-bit)**.

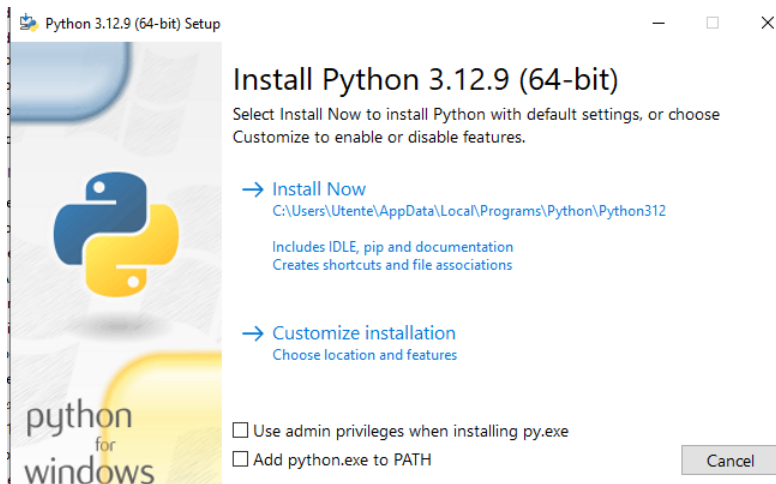
Questo avvierà direttamente il download del file .exe corretto.

6. nella cartella "Downloads" del proprio PC" fare doppio click sulla cartella:

 python-3.12.9-amd64.exe

per avviare l'installer.

7. Nel wizard dell'installer, in questo step:



fare check su “Add Python.exe to PATH” (importante) e poi scegliere l’opzione “Install Now”

8. Una volta che l’installazione è terminata chiudere il terminale e riaprirlo. Quindi digitare “python”. Si deve aprire una shell Python con la versione **3.12.9**:

```
C:\Users\Utente>python
Python 3.12.9 (tags/v3.12.9:fdb8142, Feb  4 2025, 15:27:58) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Ok, abbiamo installato python e funziona.

Si esce da python con ctrl+Z.

9. Torniamo al terminale: sotto la cartella C:\Utenti\<Utente> scaricare il software Openai col seguente comando:

```
git clone https://github.com/openai/privacy-filter.git
```

dove [git](#) è un software (molto buono e diffuso) per scaricare e condividere software.

Il risultato dev’essere:

```
C:\Users\Utente>git clone https://github.com/openai/privacy-filter.git
Cloning into 'privacy-filter'...
remote: Enumerating objects: 76, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 76 (delta 1), reused 1 (delta 1), pack-reused 72 (from 1)
Receiving objects: 100% (76/76), 104.03 KiB | 2.04 MiB/s, done.
Resolving deltas: 100% (8/8), done.
```

E’ importante che tutti i passi terminino con “done” e non ci siano errori.

10. Posizionarsi sulla nuova cartella creata al passo 9 con:

```
cd privacy-filter
```

Questa è la **cartella principale del progetto**.

Eeguire il comando `dir`.

Si dovrà ottenere qualcosa di simile (date a parte):

```
C:\Users\Utente\privacy-filter>dir
Il volume nell'unità C è OS
Numero di serie del volume: 7E81-0BB8

Directory di C:\Users\Utente\privacy-filter

01/06/2026  14:21    <DIR>          .
01/06/2026  14:21    <DIR>          ..
01/06/2026  14:21                125 .gitignore
01/06/2026  14:21            3.450 EVAL_AND_OUTPUT_MODES.md
01/06/2026  14:21    <DIR>          examples
01/06/2026  14:21            2.318 FINETUNING.md
01/06/2026  14:21            11.558 LICENSE
01/06/2026  14:21    <DIR>          opf
01/06/2026  14:21            1.536 OUTPUT_SCHEMAS.md
01/06/2026  14:21            485 pyproject.toml
01/06/2026  14:21           12.802 README.md
01/06/2026  14:21            503 SECURITY.md
               8 File                32.777 byte
               4 Directory  237.233.242.112 byte disponibili
```

In particolare, in questa cartella devono esistere:

- il file `pyproject.toml`
- il file `README.md`
- la sottocartella `opf`
- la sottocartella `examples`

11. Dalla cartella di progetto del terminale fare:

```
pip install -e .2
```

² `pip install -e .` è un comando normalissimo in Python, anche se a prima vista sembra un po' strano. Significa:

- `pip install` = installa un pacchetto Python
- `.` = il pacchetto da installare è **la cartella corrente**
- `-e` = installazione **editable**, cioè "modificabile" o "in sviluppo"

In pratica, invece di copiare il pacchetto dentro `site-packages` come una installazione tradizionale, `pip` crea un collegamento alla cartella del progetto locale. Quindi, se poi si modifica il codice sorgente in quella cartella, Python vede subito le modifiche senza dover reinstallare ogni volta. Questo uso è coerente con il repository ufficiale `openai/privacy-filter`, che contiene un progetto Python con `pyproject.toml` e nome pacchetto `opf`, quindi installabile localmente dalla directory del repo.

Nel caso specifico di **OpenAI Privacy Filter**, il comando va eseguito dopo aver clonato il repository GitHub ufficiale e dopo essersi posizionati dentro quella cartella. Il repository è effettivamente pubblico e molto recente, e non ha una release GitHub preconfezionata al momento, quindi ha senso che la guida suggerisca l'installazione dal sorgente locale invece di un semplice `pip install nomepacchetto`.

Dobbiamo ottenere un output di questo tipo:

```
Successfully built opf
Successfully installed .....
```

NB. Non si confondano questi due comandi (simili):

```
pip install .
pip install -e .
```

Il primo comando installa una copia “fissa” del pacchetto, il secondo installa il pacchetto in modalità sviluppo, collegato ai file sorgente locali ³.

Con il secondo comando, pip crea un **eseguibile** (un *file.exe*) chiamato opf, per esempio in una cartella del tipo:

```
C:\Users\Utente\AppData\Local\Programs\Python\Python312\Scripts\opf.exe
---
```

12. Verificare che opf sia stato installato correttamente in questo modo:

```
python -m pip show opf 4
```

che deve restituire qualcosa di simile a:

³ Questa modalità è molto usata quando si vuole:

- sviluppare o ispezionare il codice;
- fare prove modificando il progetto;
- evitare di reinstallare dopo ogni piccola modifica.

Per un semplice utilizzo “da utente finale”, spesso si preferisce un’installazione *non editable*; per un progetto appena pubblicato su GitHub, invece, -e è abbastanza comune.

⁴ pip show opf e where opf rispondono a **due domande diverse**.

- pip show dice dove è *registrato* il pacchetto. La riga *Location*: punta a ... \Python312\Lib\site-packages, dove stanno i metadati; con l’installazione *editable*, però, il codice sorgente vero resta nella cartella di progetto, come conferma la riga *Editable project location*: C:\Users\Utente\privacy-filter.
- where opf dice dove si trova l’*eseguibile opf.exe*, cioè in ... \Python312\Scripts. È questo il file che il PATH rende richiamabile digitando opf.

In breve: **Scripts** ospita il comando, **site-packages** la registrazione del pacchetto, la **cartella di progetto** il codice sorgente.

```
C:\Users\Utente\privacy-filter>python -m pip show opf
Name: opf
Version: 0.1.0
Summary: Standalone local-only OPF PyTorch model and eval tools.
Home-page:
Author:
Author-email:
License:
Location: C:\Users\Utente\AppData\Local\Programs\Python\Python312\Lib\site-packages
Editable project location: C:\Users\Utente\privacy-filter
Requires: huggingface_hub, numpy, packaging, safetensors, tiktoken, torch
Required-by:
```

Un output di questo tipo ci dice 4 cose importanti:

A. Le due righe:

```
Name: opf
Version: 0.1.0
```

confermano che Python vede il pacchetto opf.

B. *Summary: Standalone local-only OPF PyTorch model and eval tools.* **Cioè, che cosa è opf!!**

C. L'installazione è davvero in modalità *editable*.

La riga: `Editable project location: C:\Users\Utente\privacy-filter`

conferma che il comando: `python -m pip install -e .`

ha funzionato e che il pacchetto installato punta alla cartella locale del progetto.

D. Stiamo usando il Python ufficiale installato in Windows, come indicato dalla riga:

```
Location: C:\Users\Utente\AppData\Local\Programs\Python\Python312\Lib\site-packages
```

Quindi la situazione attuale è:

opf è installato
opf punta correttamente alla cartella C:\Users\Utente\privacy-filter
opf è installato nel Python 3.12 ufficiale di Windows

13. Verifichiamo (ulteriormente) la disponibilità dell'help (**e di opf!**) con

```
python -m opf -help
```

14. A questo punto, abbiamo **uno script python di nome opf** che si può eseguire tramite `python -m opf`

Per una **prima anonimizzazione una tantum su CPU**, cioè un primo uso del modello, digitare:

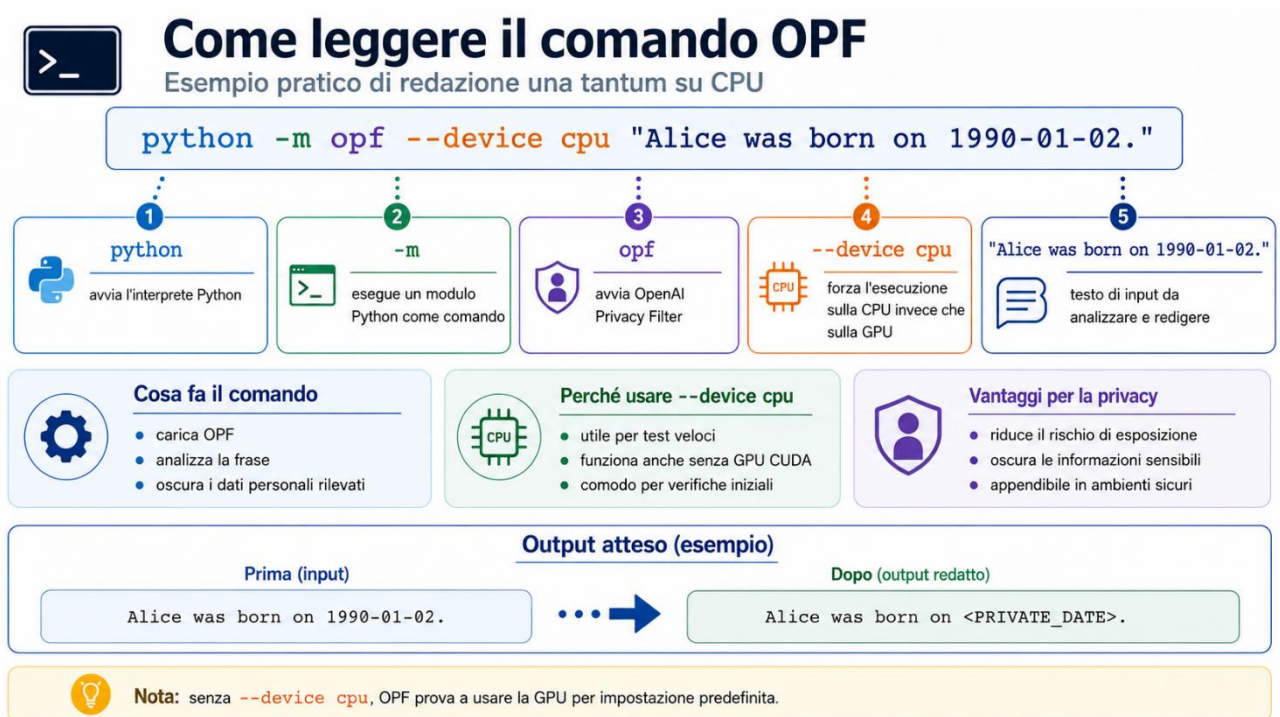
```
python -m opf --device cpu "Alice was born on 1990-01-02."
```

Si deve ottenere:

Alice was born on <PRIVATE_DATE>.

Il modello ha riconosciuto un dato sensibile, la data di nascita di Alice, e l'ha sostituito con una etichetta di tipologia!

La seguente infografica descrive tutte le componenti del precedente importante comando:



Come mai `opf` non ha riconosciuto Alice come <PRIVATE_PERSON>?

⚠ **Nota — perché "Alice" non viene oscurato.** Non è un errore: il modello lascia in chiaro il nome di battesimo isolato *Alice*, mentre maschera i nomi completi. Confronta:

- Alice was born on 1990-01-02. → Alice was born on <PRIVATE_DATE>. (nome singolo **non** oscurato)
- Mario Rossi was born on 1980-06-01. → <PRIVATE_PERSON> was born on <PRIVATE_DATE>. (nome + cognome oscurato)

Il comportamento è **coerente col design**: un nome di battesimo da solo non identifica univocamente nessuno, quindi non viene trattato come dato personale.

15. E' possibile poter usare direttamente opf (senza python -m prima), in questo modo.

Verificare dov'è opf digitando, dalla stessa cartella del terminale:

```
where opf
```

Se tutto è a posto, dovrebbe comparire qualcosa come:

```
C:\Users\Utente\AppData\Local\Programs\Python\Python312\Scripts\opf.exe
```

Ciò vuol dire che questa cartella è **nel PATH!** (where cerca nel PATH!)

Possiamo allora usare in sicurezza direttamente il comando

```
opf --device cpu "Alice was born on 1990-01-02."
```

per ottenere lo stesso risultato.

L'importante, comunque, è che funzioni il comando tramite python visto prima:

```
python -m opf --device cpu "Alice was born on 1990-01-02."
```

16. L'installazione di base del privacy filter di OpenAI funziona!

Adesso conviene procedere in questo ordine:

1. capire dove è stato scaricato il modello;
2. provare la anonimizzazione di un file intero;
3. eventualmente passare alla GPU;
4. poi, solo dopo, guardare eval e fine-tuning.

16.1 Verificare dove si trova il modello

Dopo la prima esecuzione riuscita, opf dovrebbe avere creato una cartella sotto il profilo utente:

```
C:\Users\Utente\.opf\privacy_filter
```

Controllare con (da terminale):

```
dir %USERPROFILE%\.opf\privacy_filter
```

Se vediamo un output simile al seguente:

```

C:\Users\Utente\privacy-filter>dir %USERPROFILE%\opf\privacy_filter
Il volume nell'unità C è OS
Numero di serie del volume: 7E81-0B88

Directory di C:\Users\Utente\opf\privacy_filter
23/04/2026  17:25    <DIR>        .
23/04/2026  17:25    <DIR>        ..
23/04/2026  17:21    <DIR>        .cache
23/04/2026  17:21             707 config.json
23/04/2026  17:21             4.108 dtypes.json
23/04/2026  17:25      2.798.984.088 model.safetensors
23/04/2026  17:21             372 viterbi_calibration.json
                4 File      2.798.989.275 byte
                3 Directory 235.649.200.128 byte disponibili

```

allora il checkpoint è stato scaricato correttamente.

Il file più importante è: `model.safetensors`, che pesa circa: 2.798.984.088 byte, cioè circa **2.8 GB**. Quello è il modello vero e proprio.

Quindi ora la situazione è:

opf = comando/eseguibile

C:\Users\Utente\opf\privacy_filter = cartella del modello/checkpoint

model.safetensors = file principale del modello

Si può anche verificare se esiste una variabile d'ambiente `OPF_CHECKPOINT` con il comando:

```
echo %OPF_CHECKPOINT%
```

Se stampa semplicemente:

```
%OPF_CHECKPOINT%
```

oppure una riga vuota, significa che la variabile non è impostata. Va bene: in quel caso opf usa la cartella predefinita:

```
C:\Users\Utente\opf\privacy_filter
```

16.2. Provare con un file di testo

Creare un piccolo file di prova. Ad esempio, da *cmd*:

```
echo Alice was born on 1990-01-02. She lives in Rome in Via del
Corso 120. > prova_privacy.txt
```

Verificare il contenuto del file digitando:

```
prova_privacy.txt
```

Deve aprirsi *notepad* con il contenuto.

In alternativa, è anche possibile verificare il contenuto del file con:

```
type prova_privacy.txt
```

Poi eseguire:

```
opf --device cpu -f prova_privacy.txt
```

Dobbiamo ottenere:

```
Alice was born on <PRIVATE_DATE>. She lives in Rome in <PRIVATE_ADDRESS>.
```

Entrambi i dati sensibili sono stati riconosciuti, classificati e oscurati.

Se il file non è nella cartella corrente, usare il percorso completo, per esempio:

```
opf --device cpu -f  
C:\Users\Utente\.opf\privacy_filter\prova_privacy.txt
```

Anche in questo caso si ottiene:

```
Alice was born on <PRIVATE_DATE>. She lives in Rome in <PRIVATE_ADDRESS>.
```

NB. Per salvare su file i risultati di questo esempio, fare così:

```
opf --device cpu -f prova_privacy.txt > prova_privacy_redatto.txt
```

Poi controllarne il contenuto con :

```
type prova_privacy_redatto.txt
```

Così abbiamo verificato il flusso classico:

file originale -> OPF -> file redatto

Il file originale non viene modificato.

16.3. Provare con un esempio più realistico

Creare un file txt:

```
notepad esempio_cliente.txt
```

Dentro scrivere più dati personali, per esempio:

```
Mario Rossi was born on 1980-06-01.  
He lives in Via Roma 25, Turin, Italy.  
His email is mario.rossi@example.com.  
His phone number is +39 333 1234567.  
His bank account number is IT60X0542811101000000123456.
```

Salvare il file e poi lanciare:

```
opf --device cpu -f esempio_cliente.txt
```

Si ottiene questo output:

```
C:\Users\Utente\privacy-filter>opf --device cpu -f esempio_cliente.txt  
<PRIVATE_PERSON> was born on <PRIVATE_DATE>.  
He lives in <PRIVATE_ADDRESS>.  
His email is <PRIVATE_EMAIL>.  
His phone number is <PRIVATE_PHONE>.  
His bank account number is <ACCOUNT_NUMBER>.
```

Questo esempio è utile perché **verifica più categorie**: persona, data, indirizzo, email, telefono, conto.

Come nell'esempio precedente, possiamo anche salvare il risultato con:

```
opf --device cpu -f esempio_cliente.txt >
esempio_cliente_redatto.txt
```

Poi:

```
type esempio_cliente_redatto.txt
```

17. Provare la **GPU** (se disponibile sul PC)

Occorre installare pytorch, una libreria Python open source sviluppata da Meta, usata per creare e addestrare **reti neurali artificiali** (intelligenza artificiale e deep learning).

Per **OPF / OpenAI Privacy Filter** la cosa importante è:

torchaudio non serve torchvision non serve serve solo torch
--

Ed infatti nel *pyproject.toml* di OPF le dipendenze includono *torch*, ma non richiedono né *torchvision* né *torchaudio*.

Quindi digitare questo comando;

```
python -m pip install torch --index-url
https://download.pytorch.org/whl/cu132
```

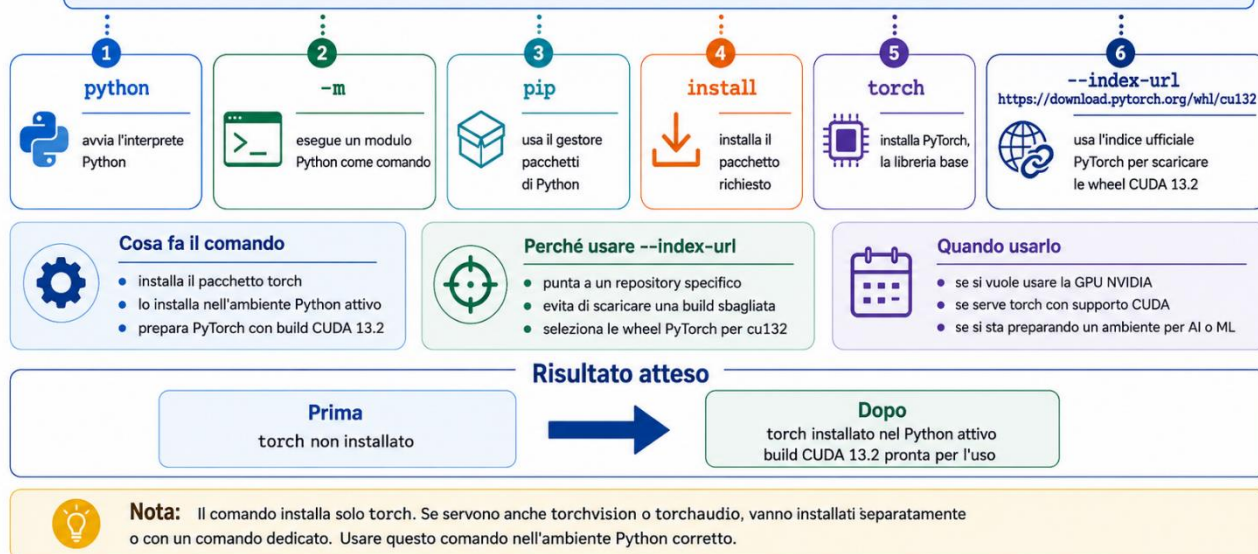
La seguente infografica descrive le componenti di questo importante comando:



Come leggere il comando di installazione PyTorch

Esempio pratico: installare torch con build CUDA 13.2

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cu132
```



👉 La versione CUDA132 è la migliore per una RTX 5090 (in subordine la 13.0), **mai la 12.6**. Il motivo è la compute capability.

Verificare la corretta installazione di torch col seguente comando:

```
python -c "import torch; print(torch.__version__);  
print(torch.version.cuda); print(torch.cuda.is_available());  
print(torch.cuda.get_device_name(0) if torch.cuda.is_available()  
else 'NO CUDA')"
```

L'obiettivo è vedere qualcosa del tipo:

2.x.x+cu130

13.0

True

NVIDIA GeForce RTX 5090

```
2.12.0+cu130  
13.0  
True  
NVIDIA GeForce RTX 5090
```

Il modello di GPU NVIDIA dipende dal singolo PC. Non è necessario avere per forza questo modello (RTX 5090, molto potente).

Se invece *cu130* non trova pacchetti compatibili, provare la build CUDA più tradizionale:

```
python -m pip install torch --index-url  
https://download.pytorch.org/whl/cu128
```

Finalmente siamo pronti per la prova della GPU.

Dobbiamo solo, prima, resettare un flag. Infatti quando opf gira su GPU, prova a usare una componente accelerata chiamata **Triton** per i kernel MoE, cioè **Mixture-of-Experts** ⁵.

Nel codice di opf, quando il modello non gira su CPU, la variabile OPF_MOE_TRITON viene considerata attiva per impostazione predefinita: default=not_running_on_cpu . Se Triton non è installato, se si esegue il prossimo comando di prova della GPU, opf genera questo messaggio:

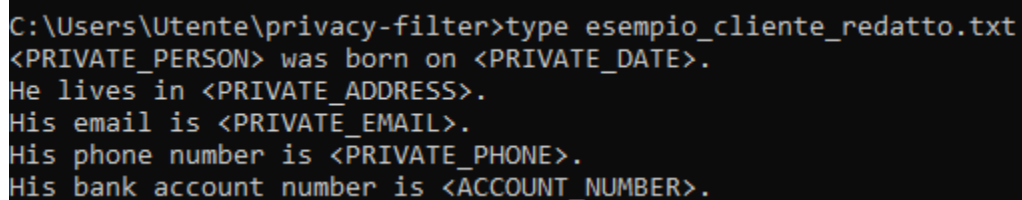
Install `triton` or unset OPF_MOE_TRITON.

La via più semplice è resettare il flag, in questo modo:

```
set OPF_MOE_TRITON=0
```

Ed ora la prova su GPU può funzionare!!

```
opf -f esempio_cliente.txt > esempio_cliente_redatto.txt
```



```
C:\Users\Utente\privacy-filter>type esempio_cliente_redatto.txt
<PRIVATE_PERSON> was born on <PRIVATE_DATE>.
He lives in <PRIVATE_ADDRESS>.
His email is <PRIVATE_EMAIL>.
His phone number is <PRIVATE_PHONE>.
His bank account number is <ACCOUNT_NUMBER>.
```

La configurazione GPU ha funzionato correttamente.

La situazione finale è questa:

opf installato correttamente modello scaricato in C:\Users\Utente\.opf\privacy_filter PyTorch CUDA funzionante Triton disattivato con OPF_MOE_TRITON=0 anonimizzazione GPU funzionante
--

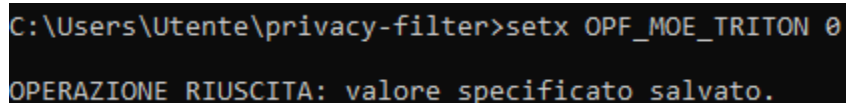
A questo punto rendiamo stabile la variabile OPF_MOE_TRITON.

set OPF_MOE_TRITON=0 infatti vale solo per la finestra *cmd* corrente. Quando si chiude il terminale, l'impostazione si perde.

Per renderla permanente per l'utente Windows fare:

```
setx OPF_MOE_TRITON 0
```

Deve dare:



```
C:\Users\Utente\privacy-filter>setx OPF_MOE_TRITON 0
OPERAZIONE RIUSCITA: valore specificato salvato.
```

⁵ **Mixture of Experts (MoE)** è una tecnica di apprendimento automatico scalabile in cui un modello di intelligenza artificiale di grandi dimensioni viene suddiviso in sottoreti più piccole e specializzate chiamate "esperti".

Per verificare ulteriormente che la modifica della variabile sia persistente, si deve chiudere e riaprire il Prompt dei comandi e verificare:

```
echo %OPF_MOE_TRITON%
```

Deve restituire:

```
C:\Users\Utente\privacy-filter>echo %OPF_MOE_TRITON%
0
```

18. Benchmark CPU vs GPU

Su quali volumi di testo da elaborare si vede la differenza tra CPU e GPU?

Facciamo un semplice benchmark.

Creiamo un file più grande, per esempio duplicando molte volte il testo di prova, per tre file separati (di 1000, 10000 e 50000 righe di test):

Da terminale *cmd*:

```
(for /L %i in (1,1,1000) do @echo Mario Rossi was born on 1980-06-01. He lives in Via Roma 25, Turin, Italy. His email is mario.rossi@example.com.) > test_1000.txt
```

```
(for /L %i in (1,1,10000) do @echo Mario Rossi was born on 1980-06-01. He lives in Via Roma 25, Turin, Italy. His email is mario.rossi@example.com.) > test_10000.txt
```

```
(for /L %i in (1,1,50000) do @echo Mario Rossi was born on 1980-06-01. He lives in Via Roma 25, Turin, Italy. His email is mario.rossi@example.com.) > test_50000.txt
```

Poi, sempre da *cmd*, eseguire questi 3 comandi, uno alla volta, annotando l'output:

TEST CPU

```
python -c "import subprocess,time; t=time.perf_counter(); f=open('out_cpu_1000.txt','w',encoding='utf-8'); r=subprocess.run(['opf','--device','cpu','-f','test_1000.txt'],stdout=f,stderr=subprocess.PIPE,text=True); f.close(); print('CPU 1000 returncode =',r.returncode); print('CPU 1000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-1000:])"
```

```
python -c "import subprocess,time; t=time.perf_counter(); f=open('out_cpu_10000.txt','w',encoding='utf-8'); r=subprocess.run(['opf','--device','cpu','-f','test_10000.txt'],stdout=f,stderr=subprocess.PIPE,text=True); f.close(); print('CPU 10000 returncode =',r.returncode); print('CPU
```

```
10000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-1000:])]"
```

```
python -c "import subprocess,time; t=time.perf_counter();  
f=open('out_cpu_50000.txt','w',encoding='utf-8');  
r=subprocess.run(['opf','--device','cpu','-  
f','test_50000.txt'],stdout=f,stderr=subprocess.PIPE,text=True);  
f.close(); print('CPU 50000 returncode =',r.returncode); print('CPU  
50000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-  
1000:])]"
```

TEST GPU

E poi i 3 comandi per la GPU, anch'essi uno per volta, annotando l'output.

```
python -c "import subprocess,time; t=time.perf_counter();  
f=open('out_gpu_1000.txt','w',encoding='utf-8');  
r=subprocess.run(['opf','-  
f','test_1000.txt'],stdout=f,stderr=subprocess.PIPE,text=True);  
f.close(); print('GPU 1000 returncode =',r.returncode); print('GPU  
1000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-  
1000:])]"
```

```
python -c "import subprocess,time; t=time.perf_counter();  
f=open('out_gpu_10000.txt','w',encoding='utf-8');  
r=subprocess.run(['opf','-  
f','test_10000.txt'],stdout=f,stderr=subprocess.PIPE,text=True);  
f.close(); print('GPU 10000 returncode =',r.returncode); print('GPU  
10000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-  
1000:])]"
```

```
python -c "import subprocess,time; t=time.perf_counter();  
f=open('out_gpu_50000.txt','w',encoding='utf-8');  
r=subprocess.run(['opf','-  
f','test_50000.txt'],stdout=f,stderr=subprocess.PIPE,text=True);  
f.close(); print('GPU 50000 returncode =',r.returncode); print('GPU  
50000 seconds =',round(time.perf_counter()-t,3)); print(r.stderr[-  
1000:])]"
```

Il valore da annotare, per ognuno di questi run, è sempre

seconds = ...

e il test è valido solo se:

```
returncode = 0
```

Alla fine avremo una tabella del tipo:

```
test_1000.txt CPU: ... sec GPU: ... sec
```

```
test_10000.txt CPU: ... sec GPU: ... sec
```

```
test_50000.txt CPU: ... sec GPU: ... sec
```

Se il file *test_50000.txt* è molto grande, la CPU potrebbe impiegare parecchio. Conviene iniziare con 1000 e 10000; e poi passare al 50000 solo dopo avere verificato **che i primi due test terminino correttamente**.

Nota. Il fatto che i file di questo stress test siano stati costruiti per ripetizione della stessa frase **non inficia il confronto prestazionale CPU/GPU**, purché l'obiettivo sia misurare la **velocità di elaborazione grezza** di OPF su volumi crescenti.

Per il benchmark prestazionale che stiamo facendo, il punto principale è confrontare:

stesso input

stesso comando

stesso modello

stesso output

CPU vs GPU

Quindi, se il file *test_1000.txt* è identico per CPU e GPU, il confronto resta valido.

La ripetizione della stessa frase può però rendere il test **meno rappresentativo di un caso reale**. In pratica: va bene per misurare throughput puro, **non è ideale per simulare documenti reali**, perché un testo reale può contenere:

- frasi di lunghezza diversa
- punteggiatura variabile
- nomi diversi
- indirizzi diversi
- email diverse
- date in formati diversi
- righe vuote
- testo non sensibile
- testo multilingue
- layout sporco da PDF/OCR

Tutto questo può influire su tokenizzazione, decodifica degli span e quantità di output prodotto.

Quindi occorre usare i tre file ripetitivi come **primo benchmark tecnico**, utile per capire quando la GPU comincia a staccare la CPU.

Poi si può fare un secondo test più realistico con testo variato.

Per esempio si può creare un file con frasi diverse, sempre da cmd:

```
(  
echo Mario Rossi was born on 1980-06-01 and lives at Via Roma 25, Turin.  
echo Alice Smith was born on 1990-01-02 and her email is  
alice.smith@example.com.  
echo Giuseppe Verdi has phone number +39 333 1234567.  
echo The customer account number is IT60X0542811101000000123456.  
echo Please contact Laura Bianchi at laura.bianchi@example.org.  
echo John lives at 10 Downing Street, London.  
echo This line contains no personal data.  
echo Meeting scheduled for 2026-06-01 with Antonio Neri.  
) > test_vario_base.txt
```

Poi lo si duplica molte volte:

```
(for /L %i in (1,1,10000) do @type test_vario_base.txt) >  
test_vario_10000.txt
```

A quel punto si confronta CPU e GPU su:

test_10000.txt

e su:

test_vario_10000.txt

Così si ottengono due misure diverse:

test_10000.txt benchmark sintetico regolare

test_vario_10000.txt benchmark più realistico

In sintesi: il test con frase ripetuta è valido per confrontare la **prestazione computazionale**. Non basta però, da solo, per stimare il comportamento su documenti veri. La buona pratica è fare entrambi: prima test sintetico controllato, poi test realistico con testo variato.

19. Usare file Office e PDF

Sinora abbiamo usato file testuali in input. Se i nostri dati da schermare sono in PDF o formato Office occorre prima estrarre il testo (es. con i package *pdftotext*, *python-docx*) e poi darlo in pipe al privacy-filter.

20. Tipologie custom

E' possibile creare tipologie di dati sensibili "custom" tramite fine-tuning del modello.